FastAPI & Apifox 学习笔记

> 声明：因为仅需掌握ml相关的与web交互部分+对后端web没有深入了解，以下写的笔记只展现了从模型web交互角度比较片面的理解（属实没办法，之前看Apifox的教程视频只能说看的懵懵懂懂一堆前后端交接的知识），可能对部分阐述没那么具体。

任务一：使用框架搭建 API 服务，定义预测接口

1. 知识理解

- 使用 FastAPI 将本地 Python 函数封装成 HTTP 接口。客户端提交输入后，服务器执行函数并返回处理结果
- `app = FastAPI()` 创建应用；`@app.post(...)` 注册请求方法和接口路径；处理函数负责接收数据、调用模型和组织响应
- FastAPI 负责接口逻辑，Uvicorn 负责启动应用并监听请求

2. 基本代码框架

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/home")
def home():
    return {"message": "ML API is running"}
```

启动命令：

```
uvicorn app:app --reload  #（终端上）

#或者加入代码：
if __name__=='__main__':
    uvicorn.run('app1:app',port=8000,reload=True)
```

第一个 `app` 对应文件 `app.py`，第二个 `app` 对应应用对象。`--reload` 用于在代码修改后自动重载；访问 `http://127.0.0.1:8000/docs` 可以使用 Swagger UI 测试接口

任务二：理解 HTTP 请求/响应，解析 JSON 并返回 JSON

1. 请求和响应的组成

- **请求方法（Method）**：表示请求的操作方式，例如 `POST` 用于提交数据进行处理。
- **URL**：指定服务地址和接口路径，例如 `http://127.0.0.1:8000/json_test`。
- **请求头（Headers）**：描述请求的附加信息，例如 `Content-Type: application/json` 表示请求体采用 JSON 格式。
- **请求体（Body）**：客户端提交的实际输入，例如 `{"Data_A":3,"Data_B":5}`。

- **响应状态码**：表示处理结果，例如 `200` 表示请求成功处理。
- **响应头**：描述返回内容，例如 `Content-Type: application/json` 表示响应体采用 JSON 格式。
- **响应体**：服务器返回的数据，例如 `{"Result":8}`。

其中，`127.0.0.1` 表示本机，`8000` 是服务端口。请求体是发给服务器的数据，响应体是服务器处理后返回的数据。

2. JSON 解析与序列化

```
from pydantic import BaseModel

class InputData(BaseModel):
    Data_A: float
    Data_B: float

@app.post("/json_test")
def json_test(data: InputData):
    result = data.Data_A + data.Data_B
    return {"Result": result}
```

- Pydantic 用来描述输入结构，FastAPI 自动完成 JSON 解析和数据校验。

任务三：读取请求头、解析表单和文件，并设置响应头

1. 请求头与响应头

`Content-Type` 用于说明请求体的格式，服务器可以读取该头部，对不同格式进行不同格式的处理

2. 普通表单与文件上传

```
#普通表单：
@app.post('/regin',tags=['注册'])
async def reg(name:str=Form(),password:str=Form()):     #与前面的请求名字对应
    print(f'username:{name},password={password}')
    #注册，实现数据库的添加操作
    return {
        'username':name
    }

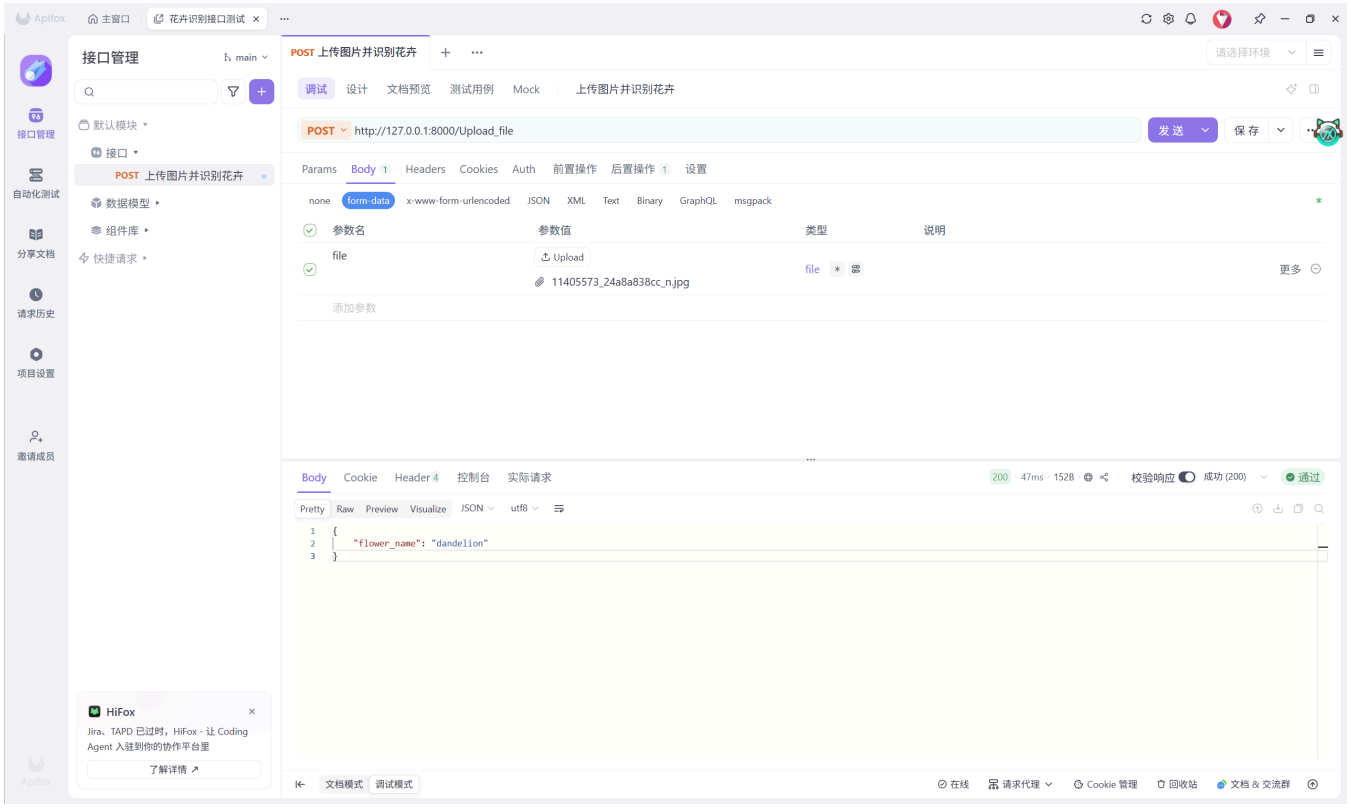
#单个文件传输：
@app.post('/file',tags=['文件传输'])
async def get_file(file:bytes=File()):
    #适合小文件上传，直接到用户内存了
    return {
        'file':len(file)
    }

#多个文件传输
@app.post('/files',tags=['文件传输'])
async def get_file(files:List[bytes]=File()):    #还有一种格式是file:UploadFile
```

```
    for file in files:
        print(len(file))
    return {
        'files':len(files)  #上传了多少个文件
    }
```

- 可以使用字节流orUploadFile两种格式（推荐使用UploadFile，保存时不会像字节流一样上传时过度占用内存）

任务四：使用 Apifox 发送请求和不同表单数据



（在完成进阶任务后使用了花卉识别模型进行发送请求和预测）

任务五：实现完整请求处理函数，并处理可能的错误

1. 按任务要求组织四则运算逻辑

```
#app.py
from fastapi import FastAPI, Form
import uvicorn
from pydantic import BaseModel, Field, field_validator


app=FastAPI()

@app.get('/home')
async def home():
    return{
        'message':'ML API function normally'
    }
```

```
#class Data(BaseModel):
    Data_A:int=Field(default=0)
    Operation:str = 'sum'
    Data_B:int=Field(default=0)



@app.post("/form_test")
async def form_test(
    Data_A: int = Form(),
    Operation: str = Form(),
    Data_B: int = Form()
):
    if Operation == "sum":
        result = Data_A + Data_B
    elif Operation == "subtract":
        result = Data_A - Data_B
    elif Operation == "multiply":
        result = Data_A * Data_B
    elif Operation == "divide":
        if Data_B == 0:
            raise ValueError("除数不能为零")
        return Data_A / Data_B
    return {"Result": result}




if __name__=='__main__':
    uvicorn.run('app:app',port=8000,reload=True)
```

2. 状态码与异常处理

- **200：请求成功。** 服务器正常返回运算或预测结果。
- **400：业务输入不合法。** 例如不支持的运算、除数为零或图片内容无效，应返回明确的错误信息。
- **422：数据校验失败。** 例如缺少必填字段或数值类型不符合要求，通常由 FastAPI 自动返回。
- **500：服务端内部异常。** 需要查看服务器日志，定位出错原因。
- **连接失败。** 先检查服务是否启动、地址和端口是否正确，此时可能没有收到 HTTP 响应。

(仅作一定程度的了解，假如真的碰到了异常状态码更多可能会用ai问出现了什么问题)

进阶任务：调用花卉识别模型

1. 代码：

```
#app1
from fastapi import FastAPI, UploadFile
```

```python
import uvicorn
from model import Mymodel
import torch
from torchvision import transforms
from PIL import Image

model=Mymodel().to('cuda')
model.load_state_dict(torch.load('flower_model1.1.pth',map_location='cuda'))
model.eval()

app=FastAPI()

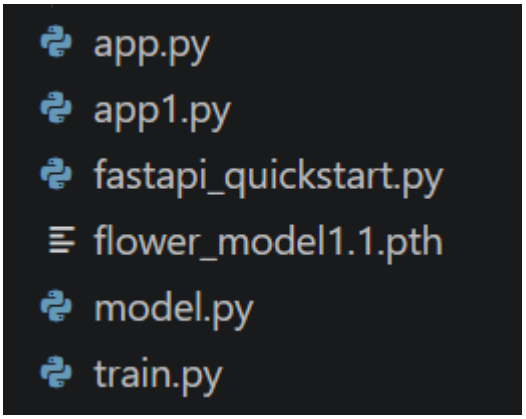
@app.get('/home')
async def home():
    return{
        'message':'ML API function normally'
    }

@app.post('/Upload_file',tags=['模型使用'])
async def use(file:UploadFile):
    print('file',file)
    image = Image.open(file.file).convert("RGB")
    file_transform=transforms.Compose([
        transforms.Resize((224,224)),
        transforms.ToTensor()
    ])
    file_t=file_transform(image)
    file_t=file_t.unsqueeze(0).to('cuda')
    with torch.no_grad():
        output=model(file_t)
        value, indice = torch.max(output,1)
        flowers=['daisy','dandelion','rose','sunflower','tulip']
        flower_name=flowers[indice.item()]
    return {'flower_name':flower_name}

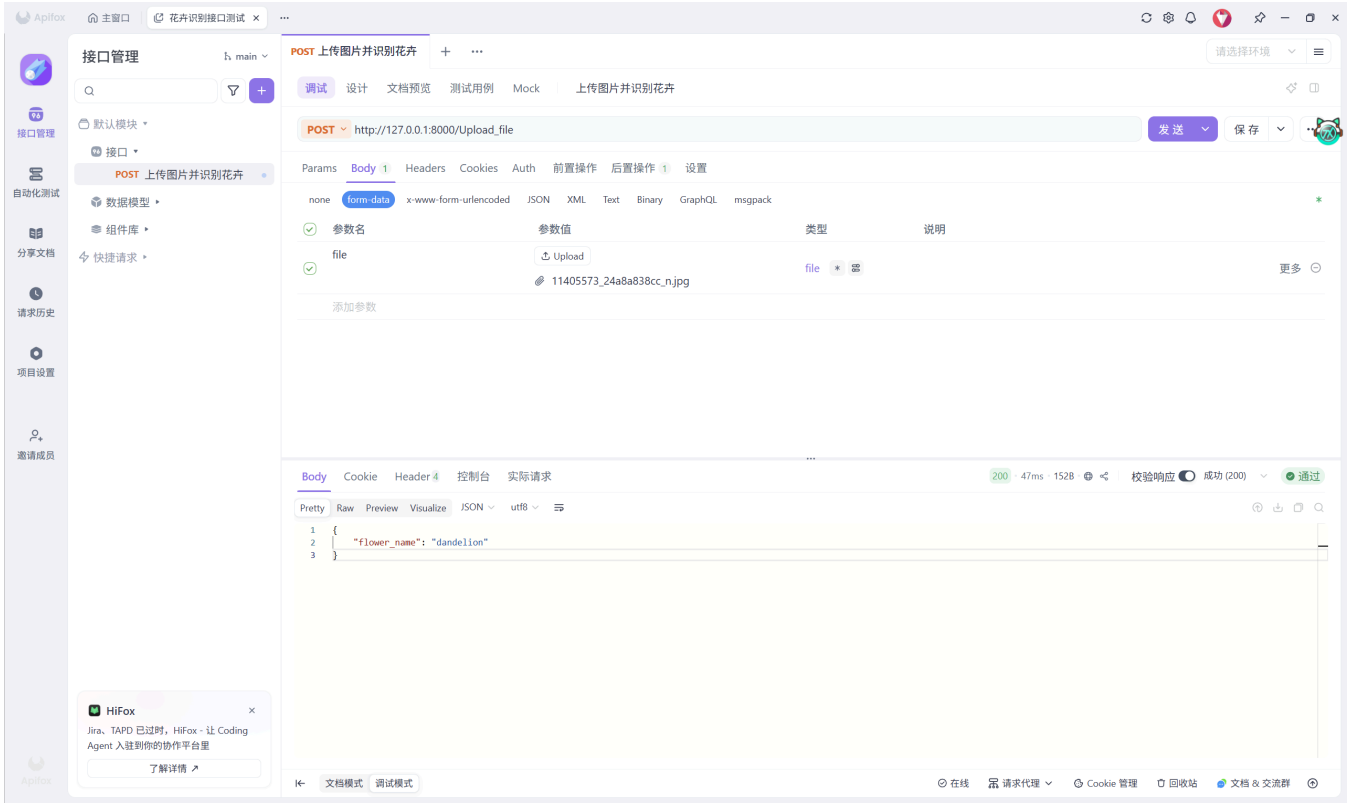
if __name__=='__main__':
    uvicorn.run('app1:app',port=8000,reload=True)
```

- `MyModel()` 创建模型结构，`load_state_dict()` 加载训练好的参数
- `model.eval()` 设置评估模式，影响 Dropout、BatchNorm 等层；`torch.no_grad()` 关闭梯度记录，两者作用不同。
- 图片大小、颜色通道和transforms方式应与训练时保持一致，否则可能影响预测结果。

文件位置：



2. Apifox调试接口（跟前面的图片相同）



3. 实践感悟

- `ToTensor()` 后单张 RGB 图片通常是 `[3,224,224]`，`unsqueeze(0)` 增加 batch 维度，变为 `[1,3,224,224]`。
- 曾遇到输入在 GPU、模型在 CPU 的错误。应同时设置 `model.to(device)` 和 `tensor.to(device)`；仅设置 `torch.load()` 的 `map_location` 不能代替移动模型。
- 时刻注意要把tensor值拿去用是加.item()!

学习参考

- FastAPI：JSON 请求体
- FastAPI：表单与文件
- FastAPI：错误处理

- **How to Deploy a Machine Learning Model with FastAPI & Uvicorn**
- Apifox：快速上手